Module 07 — Database Programming Bridge (Python ↔ MySQL)

Phase: Advanced Python
Previous module: MySQL for Full Stack Developers
Next module: HTML (shared FRONTEND)

Module Outcome:

- Architecture (script → connector → database)
- Setup and connection parameters
- Parameterized query pattern
- CRUD mapping

Prerequisites

- Local or lab MySQL with a practice database (e.g. `training_db`)
- Comfortable with `venv` / `pip` (from Modules)

What is this bridge?

So far you saved data in **files** (CSV, JSON). That works for one user on one machine. A **web application** (CampusCart, bank portal) has **many users at once** — students ordering lunch, admins updating stock. Shared data must live in a **database server** (MySQL).

Python's job: business logic — validate input, talk to MySQL, return results to the rest of the app (CLI, API, or UI) later.

Why not only files?

Files	Database
Fine for one laptop / homework	Many users, concurrent writes
You invent search/filter	SQL queries, indexes, relations
Easy to corrupt with partial writes	Transactions (`commit` / `rollback`)

The connection lifecycle (what / why)

1. **Connect** — open a session to MySQL (host, user, password, database).
2. **Cursor** — run SQL and fetch rows.
3. **Execute** — send statements (`SELECT …`).
4. **Commit** — make INSERT/UPDATE/DELETE permanent.
5. **Close** — free server resources.

SQL injection: attacker types malicious SQL into a form. **Never** build SQL with f-strings from user input. Always use **parameterized queries** (template + separate values).

Bank | Order | Hospital | CRM | Payroll

Where used?
- College / fee apps — `students` table shared by many users
- Library systems — parameterized search by author or title
- **HDFC Bank** / **PhonePe**-style backends — balances must survive concurrent writes
- **Apollo Hospitals** / CRM records — structured queries, not one CSV per clerk
- Any multi-user app — connect → cursor → execute → commit → close

---

Important Concepts

1. Architecture

Definition: Who talks to whom.
Purpose: Separates app logic from storage — the same layering most apps use (script/API → connector → database).

```
[Python Script / App]
        ↓ mysql-connector / PyMySQL
[MySQL Server — training_db]
        ↓
[students, orders, ... tables]
```

Put a labelled ASCII (and Mermaid) diagram of Python app → connector → MySQL server → table in a docstring, then print a pointer to it.

```
"""
College fee app data path (ASCII):

  Python app  -->  mysql.connector  -->  MySQL server  -->  students table
       |                 |                    |                 |
   fee_logic.py      TCP :3306            training_db         roll,name,marks

Mermaid (paste into a Markdown previewer):

  flowchart LR
    App[Python fee app] --> Conn[mysql.connector]
    Conn --> Server[MySQL server]
    Server --> Table[students table]
"""

print("See docstring above for labelled arrows.")
```

Expected Output:

```
See docstring above for labelled arrows.
```

Write `why_apps_still_need_mysql()` whose docstring explains that a connector still talks to a DB server, and print that docstring.

```
def why_apps_still_need_mysql():
    """
    Higher-level libraries may hide SQL, but your app still sends queries
    to a MySQL (or other) database server over a connection.
    Tables store and index the data.
    Same idea as mysql.connector — only the API layer changes.
    """
    return why_apps_still_need_mysql.__doc__

print(why_apps_still_need_mysql())
```

2. Setup (lab standard)

```
python -m venv venv
source venv/bin/activate
pip install mysql-connector-python
# OR: pip install PyMySQL
pip freeze > requirements.txt
```

In an activated venv with `mysql-connector-python` installed, import `mysql.connector` and print its version.

```
# Terminal first:
#   python -m venv venv && source venv/bin/activate
#   pip install mysql-connector-python
#   pip freeze > requirements.txt

import mysql.connector

print("mysql.connector version:", mysql.connector.__version__)
print("Import OK — connector installed in this venv")
```

Expected Output:

```
mysql.connector version: <installed version>
Import OK — connector installed in this venv
```

Print that this lab uses `mysql-connector-python` and the docstring comparing it with PyMySQL (pick one connector; do not mix).

```
# Optional: second throwaway venv with PyMySQL
#   python -m venv venv_pymysql && source venv_pymysql/bin/activate
#   pip install PyMySQL && pip freeze > requirements_pymysql.txt

def note_choice():
    """
    Lab standard: mysql-connector-python (Oracle connector).
    Alternate: PyMySQL (pure Python). Both use %s placeholders.
    Pick ONE for the rest of this module — do not mix in one project.
    """
    choice = "mysql-connector-python"
    print(f"Using: {choice}")
    print(note_choice.__doc__)
```

```
note_choice()
```

Expected Output:

```
Using: mysql-connector-python

    Lab standard: mysql-connector-python (Oracle connector).
    Alternate: PyMySQL (pure Python). Both use %s placeholders.
    Pick ONE for the rest of this module — do not mix in one project.
```

3. Connection parameters

```
config = {
    "host": "localhost",
    "user": "root",           # use limited user in production
    "password": "your_password",
    "database": "training_db",
    "port": 3306,
}
```

Production: store credentials in environment variables, not source code.

Note:
- Leaving connections open → server exhaustion — close in `finally` (or use a context manager).
- Hard-coded passwords → security leak — use environment variables.
- No exception handling → cryptic crash — catch `mysql.connector.Error`.

Remember: Lifecycle = connect → cursor → execute → (`commit` for writes) → close. Skip `commit` and INSERT/UPDATE vanish.

Connect with the lab config, `SELECT DATABASE()` , print the current DB, then close cursor and connection.

```
import mysql.connector

config = {
    "host": "localhost",
    "user": "root",                 # replace with lab user
    "password": "your_password",    # replace with lab password
    "database": "training_db",
    "port": 3306,
}

conn = mysql.connector.connect(**config)
print("Connected:", conn)

cursor = conn.cursor()
cursor.execute("SELECT DATABASE();")
print("Current DB:", cursor.fetchone()[0])

cursor.close()
conn.close()
print("Closed cleanly")
```

Read `MYSQL_PASSWORD` from the environment (exit if missing), connect without a hardcoded password, then close.

```
import os
import mysql.connector

password = os.environ.get("MYSQL_PASSWORD")
if not password:
    raise SystemExit("Set MYSQL_PASSWORD in the terminal first, e.g.\n"
                     "  export MYSQL_PASSWORD='your_lab_password'")

config = {
    "host": "localhost",
    "user": "root",
    "password": password,
    "database": "training_db",
    "port": 3306,
}

conn = mysql.connector.connect(**config)
print("Connected without hardcoded password")
conn.close()
```

4. Parameterized query pattern

```
# SAFE
sql = "SELECT * FROM students WHERE roll = %s"
cursor.execute(sql, (roll,))

# UNSAFE — NEVER DO THIS
sql = f"SELECT * FROM students WHERE roll = {roll}"
```

Both `mysql-connector-python` and `PyMySQL` use `%s` placeholders (not `?` like sqlite).

Note:
- String concatenation in SQL → SQL injection — use parameterized queries.
- Wrong placeholder (`?`) → driver error — use `%s` for MySQL connectors.

Common Error: Building SQL with f-strings from user input (`f"... {roll}"`) — that is the classic injection hole. Always `execute(sql, (value,))`.

Insert one Employee Management Portal student with `%s` placeholders and a tuple, `commit()`, and print `cursor.rowcount`.

```
import mysql.connector

config = {
    "host": "localhost",
    "user": "root",
    "password": "your_password",
    "database": "training_db",
    "port": 3306,
}

conn = mysql.connector.connect(**config)
cursor = conn.cursor()

sql = "INSERT INTO students (roll, name, branch, marks) VALUES (%s, %s, %s, %s)"
cursor.execute(sql, (201, "Anita", "CSE", 88.5))
conn.commit()
```

```
print("Rows inserted:", cursor.rowcount)

cursor.close()
conn.close()
```

Print a safe `WHERE roll = %s` pattern next to an unsafe f-string that expands `1 OR 1=1`, and state the lesson.

```
# Illustrative — run against lab kit 20_sql_injection_demo with a real table.
# Safe: malicious input is treated as DATA, not SQL.
# Unsafe: input like 1 OR 1=1 can return every row.

user_input = "1 OR 1=1"

safe_sql = "SELECT * FROM students WHERE roll = %s"
# cursor.execute(safe_sql, (user_input,))  # looks for roll literally '1 OR 1=1'

unsafe_sql = f"SELECT * FROM students WHERE roll = {user_input}"
# cursor.execute(unsafe_sql)  # becomes: ... WHERE roll = 1 OR 1=1 → all rows

print("Safe pattern uses %s + tuple:", safe_sql)
print("Unsafe f-string expands to:", unsafe_sql)
print("Lesson: f-string injection broke row filtering — always use %s.")
```

Expected Output:

```
Safe pattern uses %s + tuple: SELECT * FROM students WHERE roll = %s
Unsafe f-string expands to: SELECT * FROM students WHERE roll = 1 OR 1=1
Lesson: f-string injection broke row filtering — always use %s.
```

5. CRUD mapping

Operation	SQL	Python method
Create	INSERT	`execute` + `commit`
Read	SELECT	`execute` + `fetchone` / `fetchall`
Update	UPDATE	`execute` + `commit`
Delete	DELETE	`execute` + `commit`

Note:
- Forgetting `commit()` → changes not saved — commit after DML.
- `SELECT *` in production APIs → over-fetching — name columns explicitly.

Interview: Parameterized queries send SQL template and values separately so user input cannot change the query structure — the #1 SQL-injection defence in Python + MySQL.

For one roll, parameterized INSERT / SELECT / UPDATE / DELETE with `commit` after each write, and print the row after insert and after update.

```
import mysql.connector

config = {
```

```
    "host": "localhost",
    "user": "root",
    "password": "your_password",
    "database": "training_db",
    "port": 3306,
}

roll = 301
conn = mysql.connector.connect(**config)
cur = conn.cursor()

cur.execute(
    "INSERT INTO students (roll, name, branch, marks) VALUES (%s, %s, %s, %s)",
    (roll, "Vikram", "ECE", 70.0),
)
conn.commit()

cur.execute("SELECT roll, name, marks FROM students WHERE roll = %s", (roll,))
print("After INSERT:", cur.fetchone())

cur.execute("UPDATE students SET marks = %s WHERE roll = %s", (85.0, roll))
conn.commit()

cur.execute("SELECT roll, name, marks FROM students WHERE roll = %s", (roll,))
print("After UPDATE:", cur.fetchone())

cur.execute("DELETE FROM students WHERE roll = %s", (roll,))
conn.commit()
print("Deleted rows:", cur.rowcount)

cur.close()
conn.close()
```

Connect to the library seed DB and `SELECT title, author FROM books WHERE author = %s` using a tuple, then print the rows.

```
# Assumes lab seed 02_mysql_library_seed (adjust table/column names to match seed)
import mysql.connector

config = {
    "host": "localhost",
    "user": "root",
    "password": "your_password",
    "database": "library_db",   # or your seed DB name
    "port": 3306,
}

author = "Chetan Bhagat"  # example filter value — change to match seed data

conn = mysql.connector.connect(**config)
cur = conn.cursor()
cur.execute(
    "SELECT title, author FROM books WHERE author = %s",
    (author,),
)
for row in cur.fetchall():
    print(row)

cur.close()
conn.close()
```

Step-by-Step Annotated Python 3 Code

Example 0 — MySQL table setup (run once in Workbench or mysql CLI)

```
CREATE DATABASE IF NOT EXISTS training_db;
USE training_db;

CREATE TABLE IF NOT EXISTS students (
    roll INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    branch VARCHAR(50),
    marks DECIMAL(5,2)
);
```

Example 1 — Connect and test (mysql-connector-python)

```
import mysql.connector
from mysql.connector import Error

def get_connection():
    try:
        conn = mysql.connector.connect(
            host="localhost",
            user="root",
            password="your_password",
            database="training_db"
        )
        if conn.is_connected():
            print("Connected to MySQL")
        return conn
    except Error as e:
        print("Connection failed:", e)
        return None

conn = get_connection()
if conn:
    conn.close()
    print("Connection closed")
```

Example 2 — INSERT (Create) with parameters

```
import mysql.connector

conn = mysql.connector.connect(
    host="localhost", user="root", password="your_password", database="training_db"
)
cursor = conn.cursor()

insert_sql = """
INSERT INTO students (roll, name, branch, marks)
VALUES (%s, %s, %s, %s)
"""
student = (101, "Ananya", "CSE", 88.5)

try:
    cursor.execute(insert_sql, student)
    conn.commit()                    # save permanently
    print("Rows inserted:", cursor.rowcount)
```

```
except mysql.connector.Error as err:
    conn.rollback()                 # undo on failure
    print("Insert error:", err)
finally:
    cursor.close()
    conn.close()
```

Example 3 — SELECT (Read)

```
cursor = conn.cursor(dictionary=True)
cursor.execute("SELECT roll, name, marks FROM students WHERE branch = %s", ("CSE",))
for row in cursor.fetchall():
    print(f"{row['roll']} | {row['name']} | {row['marks']}")
```

Example 4 — UPDATE and DELETE

```
cursor.execute("UPDATE students SET marks = %s WHERE roll = %s", (91.0, 101))
cursor.execute("DELETE FROM students WHERE roll = %s", (999,))
conn.commit()
```

Example 5 — SQL injection risk demo (lab — never use in real apps)

```
# VULNERABLE CODE — DO NOT USE IN PRODUCTION
def login_unsafe(cursor, username, password):
    query = f"SELECT * FROM users WHERE username='{username}' AND password='{password}'"
    cursor.execute(query)

# Attacker input: username = admin' --
# Resulting SQL ignores password check

# SAFE VERSION
def login_safe(cursor, username, password):
    query = "SELECT * FROM users WHERE username = %s AND password = %s"
    cursor.execute(query, (username, password))
```

Example 6 — Student CRUD helper module

```
# db_students.py
import mysql.connector
from mysql.connector import Error

DB_CONFIG = {
    "host": "localhost",
    "user": "root",
    "password": "your_password",
    "database": "training_db",
}

def add_student(roll, name, branch, marks):
    conn = mysql.connector.connect(**DB_CONFIG)
    cursor = conn.cursor()
    try:
        cursor.execute(
            "INSERT INTO students (roll, name, branch, marks) VALUES (%s,%s,%s,%s)",
            (roll, name, branch, marks)
        )
```

```
        conn.commit()
        return True
    except Error as e:
        conn.rollback()
        print("DB Error:", e)
        return False
    finally:
        cursor.close()
        conn.close()

def list_students():
    conn = mysql.connector.connect(**DB_CONFIG)
    cursor = conn.cursor(dictionary=True)
    cursor.execute("SELECT * FROM students ORDER BY roll")
    rows = cursor.fetchall()
    cursor.close()
    conn.close()
    return rows
```

Practice Exercises

Basic

1. Connect and print MySQL server version.
   Model answer:

```
import mysql.connector

conn = mysql.connector.connect(host="localhost", user="root", password="secret")
cur = conn.cursor()
cur.execute("SELECT VERSION()")
print(cur.fetchone()[0])
cur.close()
conn.close()
```

2. Insert 3 students with parameterized INSERT.
   Model answer:

```
sql = "INSERT INTO students (name, marks) VALUES (%s, %s)"
rows = [("Priya", 88), ("Ravi", 76), ("Anita", 91)]
cur.executemany(sql, rows)
conn.commit()
```

3. SELECT all students with marks > 75.
   Model answer:

```
cur.execute("SELECT name, marks FROM students WHERE marks > %s", (75,))
for row in cur.fetchall():
    print(row)
```

Intermediate

1. Function `get_student(roll)` returning dict or None.
   Model answer:

```
def get_student(conn, roll):
    cur = conn.cursor(dictionary=True)
    cur.execute("SELECT * FROM students WHERE roll = %s", (roll,))
    row = cur.fetchone()
    cur.close()
    return row  # dict or None
```

2. Function `update_marks(roll, new_marks)`.
   Model answer:

```
def update_marks(conn, roll, new_marks):
    cur = conn.cursor()
    cur.execute("UPDATE students SET marks = %s WHERE roll = %s", (new_marks, roll))
    conn.commit()
    affected = cur.rowcount
    cur.close()
    return affected
```

3. Menu CLI: 1-Add 2-List 3-Update 4-Delete 5-Exit.
   Model answer:

```
while True:
    print("1.Add 2.List 3.Update 4.Delete 5.Exit")
    choice = input("> ")
    if choice == "1":
        name, marks = input("name: "), float(input("marks: "))
        cur.execute("INSERT INTO students (name, marks) VALUES (%s, %s)", (name, marks))
        conn.commit()
    elif choice == "2":
        cur.execute("SELECT * FROM students")
        print(cur.fetchall())
    elif choice == "3":
        roll, marks = int(input("roll: ")), float(input("marks: "))
        cur.execute("UPDATE students SET marks=%s WHERE roll=%s", (marks, roll))
        conn.commit()
    elif choice == "4":
        roll = int(input("roll: "))
        cur.execute("DELETE FROM students WHERE roll=%s", (roll,))
        conn.commit()
    elif choice == "5":
        break
```

Interview-style

1. What is SQL injection? Prevention in Python?
   Model answer:
   SQL injection is when untrusted input is concatenated into SQL and changes query meaning. Prevent it by always using parameterized queries (`%s` placeholders / bound parameters), never string-format user data into SQL.

2. Difference commit vs rollback?
   Model answer:
   `commit` permanently saves the current transaction's changes. `rollback` undoes uncommitted changes after an error so the database stays consistent.

3. What is an ORM, and how does it relate to the raw SQL you write here?
   Model answer:
   An ORM (Object-Relational Mapper) lets you work with tables via Python classes/objects and builds SQL for you. Under the hood it still uses a connection and SQL — the same server you reach with `mysql.connector`. Prefer parameterized SQL (or a trusted ORM) over string-built queries.

Mini-Project (Assignment): Student Database Manager (Python + MySQL)

Title: student_db_cli
Lab kits: 02_mysql_library_seed (SQL practice) · 20_sql_injection_demo (parameter discipline)

What to build

Requirement	Detail
Table	`students` (roll PK, name, branch, marks)
CLI	Menu-driven CRUD via a helper module
Safety	Every query parameterized (`%s`)
UX	Graceful message on duplicate roll; errors to console
Ship	`requirements.txt` + README with MySQL setup steps

Stretch

- Add `search by branch` using parameterized `LIKE` or `=`.
- Wrap multi-step updates in try/`rollback` on failure.
- Move DB password to env vars only (document in README).

Done when

- [ ] Create / Read / Update / Delete each work from the menu
- [ ] No f-string SQL for user input anywhere
- [ ] README lists create-database + run steps for a teammate
- [ ] Pushed to GitHub without passwords in source

Interview Q&A

Shared with students after concepts and the mini-project. Use for review and interview prep — questions and model answers are included.

1. Why not store all app data in JSON files on server?
2. What happens if two users update same row simultaneously?
3. Why do many teams prefer an ORM (or helpers) over hand-written SQL for everyday CRUD?
4. Where should DB passwords live in deployed apps?
5. Difference between connector and ORM?

Q1. Why connect Python to MySQL?
A: Web and enterprise apps need centralized, concurrent, relational storage with SQL querying; Python handles app logic, MySQL stores durable structured data.

Q2. What is cursor?

A: Database cursor executes SQL and retrieves results row by row or in batches; intermediary between connection and data.

Q3. Parameterized query vs string formatting?

A: Parameters send values separately from SQL, blocking injection (typed **Employee Management Portal** name or library title never becomes a command).

Q4. commit vs rollback?

A: commit makes transaction changes permanent; rollback undoes them — **HDFC Bank** transfer or **Swiggy** checkout + stock must succeed or fail together.

Q5. ORM vs raw SQL?

A: An ORM maps Python classes/objects to tables and generates SQL, which reduces boilerplate and some injection risk; raw SQL is still useful for complex reports or performance tuning.
